Software Requirements Specification

for

Minecraft Server Manager

Version 1.0.0

Prepared by Group 7

2026-03-15

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions and Definitions 1
1.3 Project Scope 1
2. Overall Description **1**
2.1 Product Perspective 2
2.2 User Classes and Characteristics 2
2.3 Operating Environment 2
2.4 Design and Implementation Constraints 2
2.5 Assumptions and Dependencies 2
3. System Features **3**
3.1 Client Features 3
3.2 Server Features 4
4. Functional Requirements **5**
4.1 Server Requirements 5
4.2 Client Requirements 6
4.3 Integration Requirements 6
4.4 Packet Requirements 6
4.5 Data Communication Requirements 7
4.6 Usability Requirements 7
4.7 System Requirements 7
5. Data Requirements **1**
5.1 Logical Data Model 1
5.2 Data Dictionary 1
6. External Interface Requirements **1**
6.1 User Interface. 1
6.2 Software Interface. 1
7. Communications Interfaces **1**
8. API Details **1**

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

Revision History

Name	Date	Reason For Changes	Version
First Draft	2026-03-15	Initial document finished	V1.0.0

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

1. Introduction

A lightweight Minecraft server management panel that allows remote monitoring and control of a Minecraft server. Built using a Java backend application and a React web client, the system supports real time telemetry updates, server administration commands, and file management through a custom packet communication protocol.

1.1 Purpose

The purpose of this system is to provide a simple and user-friendly tool for server administrators to manage a Minecraft server without requiring direct command line access. The panel allows users to monitor system performance, control server status, edit configuration files, and perform backups from a website with a graphical user interface.

1.2 Document Conventions and Definitions

Java – https://www.java.com/en/
Minecraft – https://www.minecraft.net/en-us
Minecraft Server – The Java-based game server process being managed by the system.
Client Application – The React web interface used by administrators to monitor and control the server.
Server Application – The Java backend program responsible for executing commands and gathering telemetry data.
Telemetry – Real-time system and server performance data such as CPU usage, RAM usage, storage usage, and player count.
Custom Packet Protocol – A structured communication format used for transmitting commands and telemetry between the client and server.
Operational State – The current status of the Minecraft server (e.g., startup, stopped, running, blocked).

1.3 Project Scope

The system is intended for use by individuals or small server administrators who require a simple method of managing a Minecraft server remotely. The system does not modify the internal functionality of the Minecraft game server itself, but instead interacts with it through operating system commands and file operations.

2. Overall Description

The system will be a web based minecraft server management panel that allows the users to monitor and control minecraft server remotely. The product consists of two primary components: a java backend and a web client with react that provides the user interface. The backend communicates with the client using a persistent websocket connection and a custom packet protocol. Through the client interface the users can view system telemetry such as CPU usage,

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

RAM allocation, player count and chat logs. The system also allows users to change administration operations like starting or stopping the minecraft server.

2.1 Product Perspective

School project that is intended for learning and exploring.

2.2 User Classes and Characteristics

The primary user of this system is the one who wants to view or change Minecraft system telemetry. The user is responsible for maintaining the server, monitoring system performance and adjusting configuration settings.

2.3 Operating Environment

The backend is implemented in java and runs alongside minecraft server. The client application is built using React. The client application runs on a web browser capable of running websockets for communication.

2.4 Design and Implementation Constraints

Current constraints include: navigating new development tools/ languages (Java programming, React, API)

2.5 Assumptions and Dependencies

Blackend depends on access to minecraft server directory in order to read and configure files.
Client depends on the browser that supports websocket connection.

Assuming a functional minecraft server is already installed and configured.
Assuming that the host machine allows the commands for stopping and starting the server.
Assuming that the user has valid login credentials.
Assuming the host machine has permissions to access minecraft server directory, read log files and execute start and stop commands.

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

3. System Features

3.1 Client Features

3.1.1 Authentication:

On start up
Display login form
Send credential to backend via api
On successful verification transition to Dashboard.

3.1.2 Real Time Telemetry Dashboard:

On receiving the telemetry data from backend
Update dashboard UI with real time with CPU,RAM and Storage .
Update Player list with current online users.
Update Chat section with current server log.

3.1.3 Server Command Interface:

When the user interacts with the control panel.
Based on user choice send the Start, Stop or Restart command to the server backend.
Update the current state display on server response
Gray out button that is invalid for current state.

3.1.4 Configuration Editor:

When the user selects a file.
Request file content from the backend and display in the text editor.
On “Save” click , send the modified string back to the server.
Display verification or error message if the save fails.

3.1.5 Backup & Export:

When a user requests from the backup page.
Send the create backup or export world command to the backend.
Display real time backup progress or status.

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

3.2 Server Features

3.3 Set Minecraft directory

On starting up the server for the first time
Server asks user to input directory path of Minecraft server
Server validates the directory

3.4 Read system telemetry

On scheduled telemetry event
Run bash commands to print system info (CPU, RAM, disk)
Parse bash output to get data into the program
Create telemetry packet out of telemetry data

3.5 Read Minecraft server info

On scheduled telemetry event
Run bash commands to get server info (player count, reading chat from latest.log)
Parse bash output to get data into the program
Add server info to the telemetry packet

3.6 Read/Change Minecraft server files

On command from the client
Read specific file from the Minecraft server directory
Send to client as string

On command from client
Write passed string from the client to the Minecraft server directory

3.7 Start/stop Minecraft server

On command from client
Verify command with current state
Start/stop server

3.8 Backup Minecraft server

On command from client
Run bash command to zip or tar the /world directory
Run the large file transfer service
Transfer the zipped world file to the client

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

3.9 Create client account

On menu input
Ask user for username and password for new account
Write to encrypted file

3.10 Accept client connections

On TCP connection attempt
Ask client for username/password
Verify with account files
On successful authorization, accept connection

3.11 Send telemetry data on an interval

On time interval
Gather telemetry packet from 3.6 and 3.7
Add packet to custom packet layout
Send the packet to the client

4. Functional Requirements

4.1 Server Requirements

Requirement Number	Description
REQ-SVR-001	The server shall hold an operational state.
REQ-SVR-101	The server shall ask the user for the Minecraft server directory.
REQ-SVR-201	The server shall be able to start the Minecraft server.a
REQ-SVR-202	The server shall be able to modify the server start parameter string.
REQ-SVR-203	The server shall be able to modify the server whitelist.
REQ-SVR-204	The server shall be able to zip/tar the /world directory to save a backup.
REQ-SVR-205	The server shall be able to modify the server.properties file.
REQ-SVR-301	The server shall be able to stop and restart the minecraft server safely.
REQ-SVR-302	The server shall be able to read the JVM's RAM usage and allocation.
REQ-SVR-303	The server shall be able to read the JVM's CPU usage.
REQ-SVR-304	The server shall be able to read the Minecraft server directory storage size.
REQ-SVR-305	The server shall be able to read the machine's total storage allocation.
REQ-SVR-306	The server shall be able to read the minecraft chat from /log/latest.log.
REQ-SVR-307	The server shall be able to read the current server player count.

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

4.2 Client Requirements

Requirement Number	Description
REQ-CLT-010	The (server/client) must/shall login with the correct user/password to access the panel.
REQ-CLT-020	The server must display its status for the user to see, buttons greyed out accordingly
REQ-CLT-030	The user must be able to view CPU/RAM/storage usage on the dashboard
REQ-CLT-040	The user must be able to view the chatlog and current players
REQ-CLT-050	The user must be able to view and edit server files (startup params, server properties.jar, whitelist, etc) through the panel's text editor
REQ-CLT-060	The system shall be able to export the world file at any time
REQ-CLT-070	The system shall be able to backup the server at any time

4.3 Integration Requirements

Requirement Number	Description
REQ-INT-001	The server shall create login credentials for client connections.
REQ-INT-002	The client shall accept login credentials to connect to the server.
REQ-INT-003	The server shall send telemetry to connected clients at a 5 second interval.
REQ-INT-004	The client shall be able to send control commands to the server.
REQ-INT-005	The server shall always be accepting client commands whilst running.

4.4 Packet Requirements

Requirement	Description
REQ-PCKT-001	The system shall utilize a unified packet format consisting of a fixed-size Header (Stack), a variable-length Body (Heap), and a fixed-size Tail.
REQ-PCKT-002	The client and server must utilize Bit Fields to encapsulate the Source ID, Destination ID, Command ID, and Server State into a single 32-bit (4-byte) header.
REQ-PCKT-003	The system shall transmit actual data content from the Heap for commands involving dynamic components (strings like Usernames or Mod names) rather than transmitting a memory pointer
REQ-PCKT-004	Every transmitted packet must include a 16-bit Cyclic Redundancy Check (CRC) in the Tail section to validate data integrity upon arrival at the Server PC.

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

4.5 Data Communication Requirements

Requirement	Description
REQ-DCOM-001	The system shall establish a persistent WebSocket connection between the React Client and the Java Backend to support bi-directional, real-time data flow.
REQ-DCOM-002	The Server shall only transition its operational state (OFF, IN-PROGRESS, ON) upon receiving a valid State Change Command packet from the Client.
REQ-DCOM-003	The Server shall reject any functional commands (e.g., OP, Whitelist) if the Current State bit in the packet header does not match the actual Server State.
REQ-DCOM-004	The Java Backend shall transmit live telemetry packets containing system health (RAM usage, CPU) and Minecraft instance data (Player count) to the React Client at a minimum frequency of 1Hz (once per second).

4.6 Usability Requirements

Requirement Number	Description
REQ-USE-010	The system must have a clear layout for the main menu.
REQ-USE-020	The system must have a short description when the user hovers over a menu item.
REQ-USE-030	The system must show the connection status of the client and server at all times.

4.7 System Requirements

Requirement Number	Description
REQ-STM-010	When the server is off, it must be able to be turned on.
REQ-STM-020	The server must successfully startup before it is considered on.
REQ-STM-030	You must not be able to restart the server when it is off.
REQ-STM-040	When the server is on, it must be able to be turned off or restarted.

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

5. Data Requirements

We are planning to use a Unified Packet format which is transmitted over a TCP stream . The data model consist of three primary segment

5.1 Logical Data Model

5.1.1 Header (Stack):

This is a 32 bit fixed segment which uses Bit fields to encapsulate Source (SRC-ID) , Destination
(DEST-ID) and Commands (CMD-ID) and server state (ST).It also has a payload length field
which will tell the tcp receiver how many bytes of data is being sent.

5.1.2 Body (Heap)

This will be a dynamic data field that stores the actual data content that will be passed around.
For example a file,

5.1.3 Tail (Error Check)

A fixed size segment containing a 16 - bit cyclic redundancy check (CRC-16) used to validate data
Integrity upon receiving.

5.2 Data Dictionary

5.2.1 Server State (ST):

This would be an enum holding values for STARTUP , STOPPED, RUNNING and BLOCKED.

5.2.2 Packet Object:

A java model used on the backend containing the header block , body, and the crc fields.

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

6. External Interface Requirements

6.1 User Interface.

6.1.1 **React Web Client:** A modular interface used by administration to monitor system performance and send command
6.1.2 **Server Command Line Interface:** A simple menu based command line interface for the backend to perform local operation and view packet logs.

6.2 Software Interface.

6.3 **TCP:** The server communication with the database and the minecraft process via TCP

7. Communications Interfaces

7.1 **TCP Connection:** The system established a TCP connection between the react client (frontend) and the Java program (backend)

7.2 **Packet Encoding / Decoding:**
Coverts the java packets into object which will convert it into a byte stream for transmission
It also parses the incoming TCP stream back into the packet object and validates the CRC.

8. API Details

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

CMD-ID	Command Name	Description	Body Data (Input)	Return Data (Output)
0x01	AUTH_LOGIN	Validates credentials and establishes a session.	{username: str, pass: str}	{AuthToken: str, UserID: int}
0x02	GET_TELEMETRY	Requests a single snapshot of system and game health.	null	{cpu: float, ram: float, players: int, chat: str[]}
0x10	START_SERVER	Initiates the Minecraft server startup sequence.	null	Status: 200 (OK)
0x11	STOP_SERVER	Sends a safe shutdown command to the Minecraft process.	null	Status: 200 (OK)
0x20	FETCH_FILE	Retrieves the text of a specific configuration file .	{filename: "server.properties"}	{content: "raw_text_string"}
0x21	UPDATE_FILE	Overwrites a configuration file with new data .	{filename: str, content: str}	Status: 200 (OK)

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.

0x30	TRIGGER_BACKUP	Commands the server to zip the world directory.	null	{backupID: str, status: "pending"}

Copyright © 2013 by Karl Wiegers and Seilevel. Permission is granted to use and modify this document.